

BANKING TRANSACTION PROCESSING SYSTEM USING MASTER THEOREM

Aim

To analyze the recurrence relation $T(n) = 4T(n/2) + n^2$ using the Master Theorem and determine its time complexity for a banking transaction processing system.

Problem Statement

A banking transaction processing system uses a divide-and-conquer approach to process a large number of transactions. The recurrence relation representing the algorithm is:

$$T(n) = 4T(n/2) + n^2$$

Apply the Master Theorem to:

- Identify the parameters **a**, **b**, and **f(n)**.
- Select and justify the appropriate Master Theorem case.
- Determine the time complexity.
- Analyze the algorithm's performance for large transaction volumes.

Algorithm

Step 1: Start the program.

Step 2: Read the transaction size **n**.

Step 3: Set the values:

- $a = 4$
- $b = 2$
- $f(n) = n^2$

Step 4: Calculate

$$\log_2(4)$$

Step 5: Compare **f(n)** with $n^{(\log_2 4)}$.

Step 6: Apply the appropriate Master Theorem case.

Step 7: Display the time complexity.

Step 8: Stop the program.

Source Code

```
1  #include <stdio.h>
2  #include <math.h>
3  int main()
4  {
5      int n;
6      int a = 4, b = 2;
7
8      printf("Enter transaction size (n): ");
9      scanf("%d", &n);
10
11     double logValue = log(a) / log(b);
12
13     printf("\nMaster Theorem Analysis\n");
14     printf("-----\n");
15     printf("Recurrence : T(n) = 4T(n/2) + n^2\n");
16     printf("a = %d\n", a);
17     printf("b = %d\n", b);
18     printf("f(n) = n^2\n");
19     printf("log_b(a) = %.0lf\n", logValue);
20
21     if (logValue == 2)
22     {
23         printf("\nCase 2 of Master Theorem\n");
24         printf("Time Complexity = Theta(n^2 log n)\n");
25     }
26     printf("\nAnalysis:\n");
27     printf("The divide-and-conquer algorithm efficiently processes\n");
28     printf("large banking transactions with complexity Theta(n^2 log n).\n");
29     return 0;
30 }
```

Input and Output

```
Enter transaction size (n): 1024

Master Theorem Analysis
-----
Recurrence : T(n) = 4T(n/2) + n^2
a = 4
b = 2
f(n) = n^2
log_b(a) = 2

Case 2 of Master Theorem
Time Complexity = Theta(n^2 log n)

Analysis:
The divide-and-conquer algorithm efficiently processes
large banking transactions with complexity Theta(n^2 log n).

=== Code Execution Successful ===
```

Performance Analysis

The algorithm divides the transaction data into **4 smaller recursive problems**.

Each recursive level performs **n^2** work for processing and combining the results.

The recursion continues for **$\log n$** levels.

Therefore, the total running time becomes

$$\Theta(n^2 \log n)$$

For large banking transaction volumes, the divide-and-conquer approach helps organize the computation efficiently. However, because each level performs **n^2** work, the execution time increases as the number of transactions grows. Even so, the algorithm scales in a predictable manner and is suitable for processing large datasets.

Applications

- Banking transaction processing
- Merge Sort analysis
- Parallel computing
- Database query processing
- Large-scale data processing
- Divide-and-conquer algorithms

Advantages

- Easy to analyze recursive algorithms.
- Provides accurate time complexity.
- Useful for divide-and-conquer techniques.
- Helps compare algorithm efficiency.
- Widely used in computer science.

Limitations

- Applicable only to standard recurrence relations.
- Cannot analyze every recursive algorithm.
- Regularity conditions must be satisfied for some cases.

Result

The recurrence relation

$$T(n) = 4T(n/2) + n^2$$

was analyzed using the Master Theorem.

The parameters are:

- $a = 4$
- $b = 2$
- $f(n) = n^2$

Since

$$f(n) = \Theta(n^{(\log_2 4)}) = \Theta(n^2)$$

the recurrence satisfies **Case 2** of the Master Theorem.

Hence, the time complexity of the banking transaction processing algorithm is:

$$\Theta(n^2 \log n)$$

Thus, the algorithm is efficient for handling large transaction volumes using the divide-and-conquer approach.